PLAN DE DESARROLLO

Programas de Visión por Computadora + IA Local

2 Proyectos · Gestos · Modelos Locales · Multi-Agente

Contenido del documento

- **PROGRAMA 1 — GestureOS: Ratón y Teclado por Gestos**
- **PROGRAMA 2 — VoiceAgent: IA Multi-Agente por Voz y Gesto**

GestureOS

Control de Ratón y Teclado Virtual mediante Gestos de Mano

1. Visión General del Proyecto

GestureOS convierte tu cámara web en un dispositivo de entrada completo. Con solo mover los dedos puedes controlar el cursor del ratón sin tocar el hardware, y con un gesto específico aparece un teclado virtual flotante sobre tu vista de cámara. Todo funciona con modelos locales — sin enviar nada a servidores externos.

Concepto central: Una sola aplicación con dos modos — MODO RATÓN y MODO TECLADO — entre los que se alterna con el gesto de los dos puños cerrados simultáneos.

1.1 Los dos modos de operación

MODO RATÓN	La mano derecha actúa como ratón. El índice extendido mueve el cursor. Gestos específicos activan clics, scroll y arrastrar.
MODO TECLADO	Aparece un panel dividido: mitad superior = vista de cámara + indicadores; mitad inferior = teclado virtual QWERTY. El dedo índice toca las teclas. Se escribe en cualquier campo activo del sistema.
TRANSICIÓN	Ambos puños cerrados visibles en cámara durante ~1 segundo → toggle entre los dos modos. Un indicador visual y sonoro confirma el cambio.

2. Arquitectura de Estados (FSM)

El programa se modela como una Máquina de Estados Finitos con 4 estados principales:

IDLE → MODO RATÓN ⟷ MODO TECLADO → IDLE

Estado	Descripción	Transición hacia
IDLE / Standby	Cámara encendida pero sin detección activa. El procesamiento es mínimo.	MODO RATÓN (mano detectada)
MODO RATÓN	Se rastrea la mano derecha. El índice controla el cursor en tiempo real.	MODO TECLADO (2 puños 1 seg)

MODO TECLADO	Panel completo visible. Ambas manos disponibles para teclear.	MODO RATÓN (2 puños 1 seg)
PAUSA	Ninguna mano visible durante >3 segundos. Cursor se congela.	MODO RATÓN (mano reaparece)

3. Tabla Completa de Gestos

3.1 Modo Ratón

Gesto	Acción	Modo
Índice extendido (resto cerrado)	Mueve el cursor del ratón	RATÓN
Índice + pulgar: pinch rápido	Click izquierdo	RATÓN
Índice + pulgar: pinch mantenido	Click derecho	RATÓN
Índice + medio extendidos + descenso	Scroll hacia abajo	RATÓN
Índice + medio extendidos + ascenso	Scroll hacia arriba	RATÓN
Mano abierta → cerrar → arrastrar	Drag & Drop (agarrar y mover)	RATÓN
Dos puños cerrados (ambas manos, 1s)	Cambiar a MODO TECLADO	RATÓN

3.2 Modo Teclado

Gesto	Acción	Modo
Índice extendido → apunta a tecla	Hover / resaltar tecla	TECLADO
Índice + pulgar: pinch sobre tecla	Pulsar tecla (envía tecla al SO)	TECLADO
Índice apunta BACKSPACE	Borrar último carácter	TECLADO
Índice apunta SPACE	Espacio	TECLADO
Índice apunta ENTER	Intro / confirmar	TECLADO
Índice apunta SHIFT + tecla	Mayúscula / símbolo	TECLADO
Gesto mano cerrada + abrir (shake)	Limpiar campo de texto actual	TECLADO

Dos puños cerrados (ambas manos, 1s)	Volver a MODO RATÓN (minimiza panel)	TECLADO

4. Pipeline de Detección de la Mano

Cada frame de la cámara pasa por las siguientes etapas en orden:

#	Etapa	Tarea	Herramienta
1	**Captura**	Leer frame de webcam (BGR → RGB)	OpenCV
2	**Flip horizontal**	Efecto espejo para naturalidad	OpenCV
3	**Detección manos**	Localizar una o dos manos + 21 landmarks por mano	MediaPipe Hands
4	**Clasificación gesto**	Analizar ángulos y distancias entre landmarks → nombre del gesto	Lógica propia / modelo RF
5	**Suavizado**	Filtro Kalman o media móvil sobre coordenadas → evitar temblor	NumPy / filterpy
6	**Mapeo**	Coordenada normalizada [0,1] → coordenada pantalla en píxeles	PyAutoGUI / pyScreenInfo
7	**Acción del SO**	Mover cursor, hacer clic, scroll, escribir tecla	PyAutoGUI / pynput
8	**Render UI**	Dibujar landmarks, gestos y estado sobre el frame	OpenCV / Pygame

5. Diseño del Panel — Modo Teclado

Cuando se activa el modo teclado, la ventana se transforma en un panel de dos zonas claramente diferenciadas:

ZONA SUPERIOR (40% pantalla)	ZONA INFERIOR (60% pantalla)
• Vista de cámara en vivo (espejo) • Landmarks de las manos dibujados • Indicador del gesto actual • Texto en construcción (preview) • Icono de modo activo (TECLADO/RATÓN)	• Teclado QWERTY completo • Fila de números 1-0 • Teclas especiales: SHIFT, BACKSPACE, SPACE, ENTER • Tecla activa se ilumina al hover • Animación de pulsación al hacer click

5.1 Algoritmo de detección de pulsación de tecla

Para determinar si el dedo está 'tocando' una tecla se usa una lógica de proximidad en 2D con debounce temporal:

- Calcular distancia euclidiana entre punta del índice y centro de cada tecla visible
- Si distancia < umbral (p.ej. 30px) → estado HOVER (resaltar tecla)
- Si en HOVER y se detecta pinch (pulgar+índice) → PULSACIÓN
- Debounce de 300ms por tecla: no puede volver a pulsarse la misma tecla hasta que pase ese tiempo
- Al pulsar: enviar keystroke al SO via pynput, reproducir sonido corto, animación visual

6. Stack Técnico — Programa 1

Componente	Herramienta / Librería	Nota
Captura de video	OpenCV (cv2)	Ligero, rápido, soporte webcam directo
Detección de manos	MediaPipe Hands (Google, local)	21 landmarks 3D por mano a ~30fps
Clasificación gestos	Lógica angular propia + Random Forest	Modelo entrenado localmente, sin red
Suavizado cursor	Filtro Kalman (filterpy)	Elimina temblor del movimiento natural
Control del SO	PyAutoGUI + pynput	Mover cursor, clics, escribir teclas
Interfaz / render	OpenCV + Pygame (overlay)	Dibujar teclado y landmarks en tiempo real
Sonido feedback	pygame.mixer	Clic sonoro al pulsar tecla
Detección pantalla	screeninfo / pygetwindow	Resolución real de la pantalla del usuario

7. Fases de Desarrollo — Programa 1

FASE 1	Infraestructura base	2-3 días

- Apertura de cámara con OpenCV y visualización del stream
- Integrar MediaPipe Hands y dibujar los 21 landmarks
- Mostrar coordenadas normalizadas en pantalla para depuración

FASE 2	Modo Ratón básico	3-4 días

- Mapear coordenada del índice a coordenada de pantalla
- Implementar filtro Kalman para suavizar el movimiento
- Detectar pinch para click izquierdo
- Detectar pinch largo para click derecho

FASE 3	Sistema de gestos	4-5 días

- Implementar clasificador de gestos (ángulos de articulaciones)
- Detectar scroll up/down con dos dedos
- Detectar drag & drop con mano cerrada
- Añadir debounce y cooldown para evitar gestos duplicados
- Detección del gesto de transición (dos puños)

FASE 4	Teclado virtual	5-7 días

- Diseñar layout QWERTY con teclas como objetos con posición y tamaño
- Renderizar teclado sobre el frame con OpenCV o Pygame
- Implementar hover detection por proximidad del dedo
- Implementar pulsación con pinch + debounce
- Enviar keystrokes al SO con pynput
- Animaciones visuales y sonido de feedback

FASE 5	Pulido y rendimiento	2-3 días

- Optimizar para mantener >25fps constante
- Calibración automática al iniciar (ajuste de zona de movimiento)
- Modo de configuración para ajustar sensibilidad
- Manejo de condiciones de luz difícil
- Empaquetado como ejecutable standalone

VoiceAgent

IA Multi-Agente Activada por Voz y Gesto de Mano

1. Visión General del Proyecto

VoiceAgent es un asistente de IA completamente local que escucha cuando tú decides. Con un gesto concreto de la mano, activas el micrófono. Hablas con naturalidad: 'crea dos sub-agentes, uno que busque fotos de perros y las guarde en una carpeta, y otro que verifique que realmente son fotos de perros'. El sistema interpreta el comando, descompone la tarea en sub-agentes especializados, los lanza en paralelo y te informa del resultado.

Todo el procesamiento de voz y el orquestador central corren en local. Los agentes pueden usar APIs externas solo para tareas específicas (búsqueda de imágenes), pero el cerebro del sistema nunca sale de tu máquina.

Modelo de voz	Whisper (local, OpenAI open-source) — STT sin internet requerido
Modelo de lenguaje	Ollama local (ej: llama3, mistral, qwen2) — orquestador y agentes
Gestos	MediaPipe Hands (misma base que Programa 1) — activación del micrófono y controles
Multi-agente	Framework propio de orquestación + herramientas especializadas por agente

2. Flujo Completo de una Interacción

El ciclo de vida de una tarea sigue estos pasos numerados:

N°	Paso	Detalle
1	**Detección gesto de activación**	MediaPipe detecta el gesto asignado (ej: mano abierta, todos los dedos extendidos). Indicador visual verde 'ESCUCHANDO' aparece en pantalla.
2	**Grabación de audio**	Se activa el micrófono. Se graba hasta detectar silencio >1.5s o gesto de stop. Indicador de nivel de audio visible en tiempo real.
3	**Transcripción (STT)**	El audio se pasa a Whisper local. Devuelve texto en ~0.5-2 segundos según el modelo (tiny/base/small).

4	**Orquestación LLM**	El texto transcrito se envía al LLM local (Ollama). El LLM interpreta la intención, identifica el número de sub-agentes necesarios, su rol y sus parámetros.
5	**Creación de sub-agentes**	El orquestador instancia los agentes definidos por el LLM. Cada agente recibe: nombre, rol, herramientas disponibles, parámetros específicos.
6	**Ejecución paralela**	Los sub-agentes corren en threads/procesos separados. Pueden ejecutar tareas: búsqueda web, descarga de archivos, análisis de imágenes, operaciones de sistema.
7	**Comunicación inter-agentes**	Los agentes pueden enviarse mensajes entre sí a través de una cola interna. Ej: Agente Buscador → entrega lista de imágenes → Agente Verificador.
8	**Informe de resultados**	Cuando todos los agentes terminan, el orquestador recopila los resultados y los muestra en pantalla + TTS local opcional (pyttsx3).

3. Arquitectura Multi-Agente

3.1 Agentes del sistema — ejemplo del comando 'fotos de perros'

Cuando el usuario dice 'crea dos sub-agentes, uno que busque fotos de perro y las guarde en una carpeta, y el otro que se asegure de que son fotos de perro', el orquestador crea:

Agente	Rol	Herramientas / APIs
ORQUESTADOR	Interpreta el comando, descompone en sub-tareas, crea y supervisa los agentes. Decide cuando todos han terminado y consolida el informe.	LLM local (Ollama) + sistema de colas
Agente Buscador	Busca imágenes de perros en internet o en buscadores de imágenes. Descarga las N primeras coincidencias. Las guarda en la carpeta de destino definida.	SerpAPI / DuckDuckGo API / requests + BeautifulSoup, Pillow para guardar imágenes
Agente Verificador	Recibe la lista de archivos de imagen descargados. Carga cada imagen y usa un modelo de visión local (CLIP o clasificador) para confirmar que contiene un perro. Genera un reporte de cuáles pasaron.	CLIP local (openai/clip-vit-base) o YOLOv8 con clase 'dog', Pillow, numpy
Agente Organizador (opcional)	Mueve o renombra las imágenes verificadas positivamente. Elimina las que no sean de perros. Genera un log con el resumen.	shutil, os, json

3.2 Protocolo de comunicación entre agentes

Los agentes se comunican a través de un BUS DE MENSAJES interno (una cola Python thread-safe). Cada mensaje tiene la estructura:

{ sender, receiver, type, payload, timestamp }

- type: TASK_ASSIGN — el orquestador asigna trabajo a un agente
- type: TASK_RESULT — un agente reporta su resultado al orquestador
- type: DATA_TRANSFER — un agente pasa datos a otro (ej: lista de archivos)
- type: STATUS_UPDATE — un agente informa su progreso (0-100%)
- type: ERROR — un agente reporta fallo y solicita instrucciones

3.3 Gestos de control del sistema

Gesto	Acción	Modo
Mano abierta (5 dedos)	ACTIVAR micrófono — empezar a escuchar	STANDBY
Puño cerrado rápido	PARAR grabación y procesar comando ahora	ESCUCHANDO
Índice+medio (V de paz)	PAUSAR todos los agentes en ejecución	EJECUTANDO
Pulgar arriba	CONFIRMAR / aprobar acción pendiente	CONFIRMACIÓN
Pulgar abajo	CANCELAR tarea actual y detener agentes	EJECUTANDO
Mano abierta + movimiento circular	VER historial de comandos y resultados	CUALQUIERA

4. Diseño del Prompt del Orquestador

El núcleo del sistema es el prompt que el LLM local recibe. Debe estar estructurado para que el modelo genere siempre JSON válido con la definición de los agentes:

SYSTEM PROMPT (simplificado):

Eres un orquestador de agentes IA. El usuario te dará un comando en lenguaje natural.
Debes responder SOLO con JSON con este formato:

```
{ "agents": [ { "name": "...", "role": "...", "tools": [...], "params": {...} } ], "execution_order": "parallel|sequential", "success_condition": "..." }
```

Herramientas disponibles: web_image_search, file_save, image_classifier, file_organizer, shell_command, web_search_text

5. Stack Técnico — Programa 2

Componente	Herramienta / Librería	Nota
Captura vídeo / gestos	OpenCV + MediaPipe Hands	Misma base que Programa 1
Speech-to-Text	faster-whisper (local)	Versión optimizada de Whisper, ~3x más rápido
LLM Orquestador	Ollama (llama3.2 / mistral / qwen2.5)	100% local, API REST en localhost:11434

Ejecución multi-agente	Python threading / asyncio	Agentes en hilos paralelos con cola de mensajes
Búsqueda de imágenes	DuckDuckGo Images API / SerpAPI	Scraping de resultados de búsqueda
Clasificación imágenes	CLIP (ViT-B/32 local) o YOLOv8	Verificar que la imagen contiene un perro
Operaciones de archivo	Python os / shutil / pathlib	Guardar, mover, renombrar archivos
Text-to-Speech (opcional)	pyttsx3 / Coqui TTS	Respuesta de voz del sistema, 100% local
Interfaz	tkinter / Pygame overlay	Panel de estado con agentes activos y progreso

6. Fases de Desarrollo — Programa 2

FASE 1	Módulo de voz y activación por gesto	3-4 días

- Integrar MediaPipe para detectar el gesto de activación de micrófono
- Capturar audio con sounddevice/pyaudio hasta silencio o gesto stop
- Integrar faster-whisper para transcripción local
- Mostrar en pantalla: estado (STANDBY/ESCUCHANDO/PROCESANDO)

FASE 2	Orquestador LLM local	4-5 días

- Configurar Ollama con el modelo elegido (llama3 o mistral recomendados)
- Diseñar y refinar el system prompt del orquestador
- Parser del JSON devuelto por el LLM con validación de esquema
- Manejo de errores cuando el LLM devuelve JSON malformado

FASE 3	Sistema de sub-agentes	5-6 días

- Framework base: clase Agent con métodos run(), report(), receive_message()
- Bus de mensajes con cola thread-safe
- Agente Buscador: búsqueda y descarga de imágenes
- Agente Verificador: clasificación con CLIP o YOLOv8
- Agente Organizador: operaciones de archivo
- Sistema de progreso y logs en tiempo real

FASE 4	Gestos de control + UI	3-4 días

- Añadir gestos de pausa, cancelar, confirmar sobre los agentes
- Panel de estado visual (qué agente está activo, su progreso, sus resultados)

- Historial de comandos anteriores navegable por gesto
- Sonidos de feedback para activación de micrófono y fin de tarea

FASE 5	Generalización y robustez	4-5 días

- Hacer el sistema de herramientas extensible (añadir nuevos agentes fácilmente)
- Modo de depuración: ver el JSON del orquestador en pantalla
- TTS opcional: el sistema habla los resultados
- Documentar cómo añadir nuevas herramientas y tipos de agentes
- Pruebas con comandos complejos: 3+ agentes, dependencias entre ellos

RESUMEN EJECUTIVO

GestureOS	VoiceAgent
Dificultad: Media-Alta Tiempo estimado: 3-4 semanas Dependencia clave: MediaPipe + pynput Riesgo principal: Latencia del teclado virtual	Dificultad: Alta Tiempo estimado: 4-6 semanas Dependencia clave: Ollama + faster-whisper Riesgo principal: Calidad del JSON del LLM
Orden recomendado: Programa 1 primero El Prog. 1 construye la base de detección de gestos y MediaPipe que el Prog. 2 puede reutilizar directamente.	**Sinergia entre proyectos** Ambos comparten el módulo de gestos. Podrías incluso unirlos en una sola app que combine control de ratón + agentes de IA.